

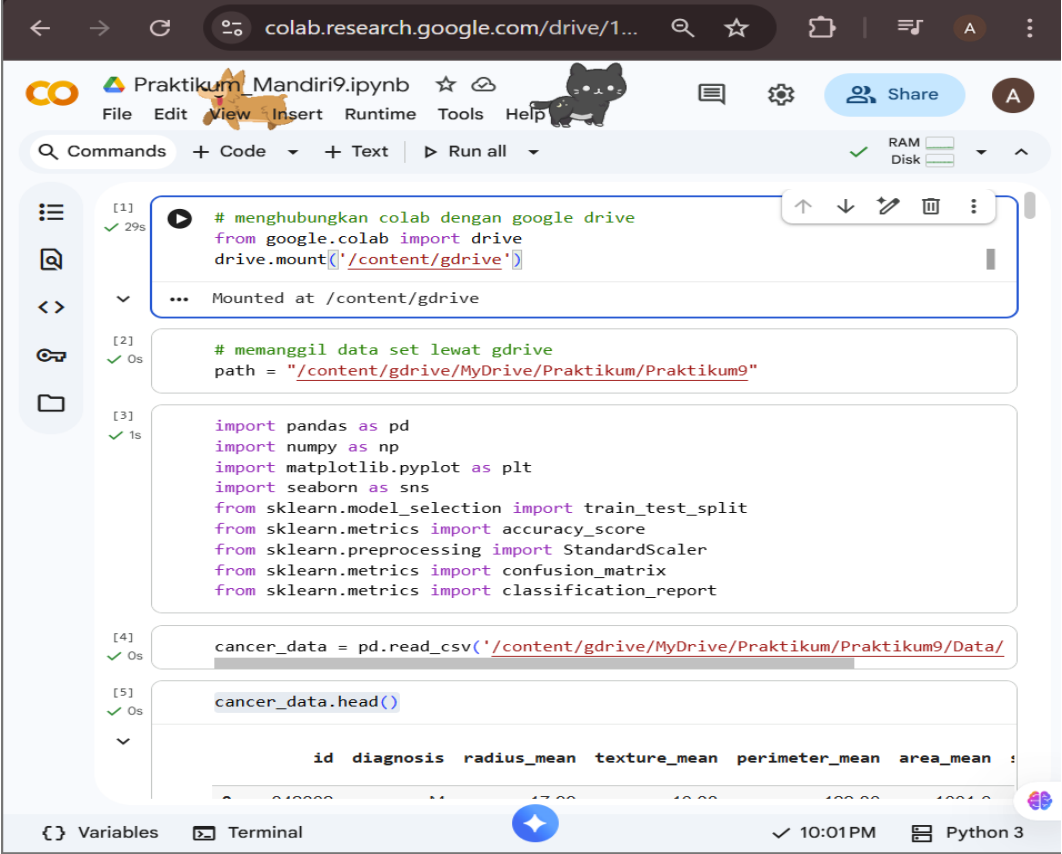
Tugas 9: Tugas Praktikum Mandiri 9 – Machine Learning

Alia Maisyarah 1 - 0110224095 ^{1*}

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 40110224095@student.nurulfikri.ac.id – email mahasiswa 1

1. Latihan Mandiri 9



```
[1] # menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/gdrive')

... Mounted at /content/gdrive

[2] # memanggil data set lewat gdrive
path = "/content/gdrive/MyDrive/Praktikum/Praktikum9"

[3]
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

[4] cancer_data = pd.read_csv('/content/gdrive/MyDrive/Praktikum/Praktikum9/Data/

[5] cancer_data.head()

id diagnosis radius_mean texture_mean perimeter_mean area_mean :
```

1. # Menghubungkan Google Colab dengan Google Drive
from google.colab import drive # Mengimpor modul untuk akses Google Drive
drive.mount('/content/gdrive') # Memasang/mount Google Drive ke direktori agar
bisa dibaca dan ditulis
2. # Memanggil dataset lewat Google Drive
path = "/content/gdrive/MyDrive/Praktikum/Praktikum9"
3. # import pandas as pd
Pandas digunakan untuk membaca, menampilkan, dan mengolah data dalam bentuk
tabel (DataFrame)

import numpy as np
NumPy digunakan untuk operasi matematis dan array numerik

```
import matplotlib.pyplot as plt
# Matplotlib digunakan untuk membuat grafik atau visualisasi seperti line plot, bar plot, histogram dll.
```

```
import seaborn as sns
# Seaborn adalah library visualisasi yang membuat grafik lebih informatif dan terlihat lebih rapi
```

```
from sklearn.model_selection import train_test_split
# Digunakan untuk membagi dataset menjadi data train (latih) dan test (uji)
```

```
from sklearn.metrics import accuracy_score
# Menghitung akurasi model machine learning
```

```
from sklearn.preprocessing import StandardScaler
# Melakukan standardisasi data sehingga fitur memiliki nilai mean = 0 dan standar deviasi = 1
```

```
from sklearn.metrics import confusion_matrix
# Membuat confusion matrix untuk mengevaluasi performa model klasifikasi
```

```
from sklearn.metrics import classification_report
# Menampilkan laporan evaluasi model (precision, recall, f1-score, support)
```

```
4. #cancer_data
pd.read_csv('/content/gdrive/MyDrive/Praktikum/Praktikum9/Data/data.csv')
# Membaca file data.csv dan menyimpannya ke dalam variabel cancer_data dalam bentuk DataFrame
# DataFrame ini nantinya akan digunakan untuk proses analisis, preprocessing, hingga model machine learning
```

The screenshot shows a Jupyter Notebook with the following content:

```
[5] ✓ 0s
cancer_data.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
0	842302	M	17.99	10.38	122.80	1001.0
1	842517	M	20.57	17.77	132.90	1326.0
2	84300903	M	19.69	21.25	130.00	1203.0
3	84348301	M	11.42	20.38	77.58	386.1
4	84358402	M	20.29	14.34	135.10	1297.0

5 rows × 33 columns

```
[6] ✓ 0s
cancer_data.tail()
```

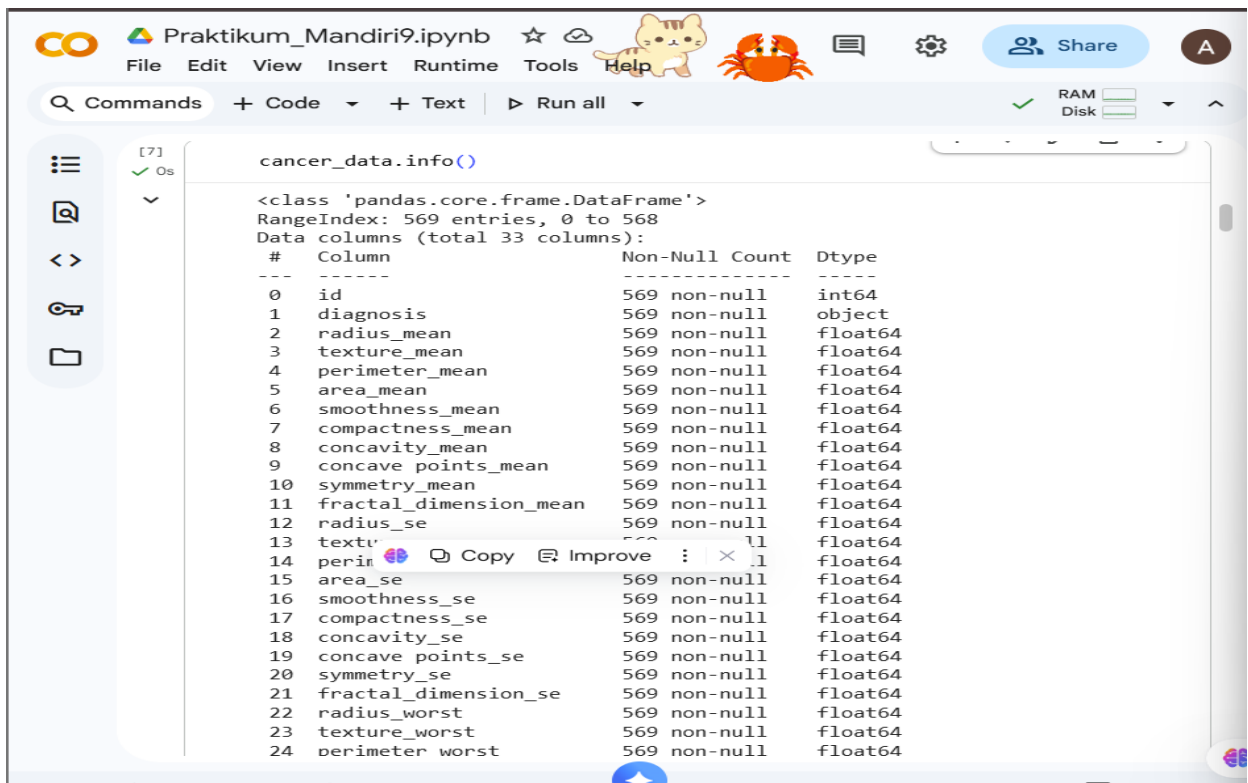
	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
564	926424	M	21.56	22.39	142.00	1479.0
565	926682	M	20.13	28.25	131.20	1261.0
566	926954	M	16.60	28.08	108.30	858.1
567	927241	M	20.60	29.33	140.10	1265.0

5. cancer_data.head()

- # Menampilkan 5 baris pertama dari dataset cancer_data
- # Fungsi ini digunakan untuk melihat isi awal dataset,
- # seperti nama kolom, contoh data, format nilai, dan memastikan
- # data sudah terbaca dengan benar.

6. cancer_data.tail()

- # Menampilkan 5 baris terakhir dari dataset cancer_data
- # Fungsi ini digunakan untuk melihat data bagian akhir,
- # memastikan apakah struktur data konsisten sampai baris terakhir
- # serta mengecek contoh nilai di bagian bawah dataset.



```
cancer_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                     Non-Null Count  Dtype  
---  --
0   id                                         569 non-null    int64  
1   diagnosis                                 569 non-null    object  
2   radius_mean                              569 non-null    float64 
3   texture_mean                             569 non-null    float64 
4   perimeter_mean                           569 non-null    float64 
5   area_mean                                569 non-null    float64 
6   smoothness_mean                          569 non-null    float64 
7   compactness_mean                         569 non-null    float64 
8   concavity_mean                           569 non-null    float64 
9   concave points_mean                      569 non-null    float64 
10  symmetry_mean                             569 non-null    float64 
11  fractal_dimension_mean                   569 non-null    float64 
12  radius_se                                 569 non-null    float64 
13  texture_worst                            569 non-null    float64 
14  perimeter_worst                          569 non-null    float64 
15  area_se                                  569 non-null    float64 
16  smoothness_se                             569 non-null    float64 
17  compactness_se                           569 non-null    float64 
18  concavity_se                             569 non-null    float64 
19  concave points_se                        569 non-null    float64 
20  symmetry_se                              569 non-null    float64 
21  fractal_dimension_se                     569 non-null    float64 
22  radius_worst                             569 non-null    float64 
23  texture_worst                            569 non-null    float64 
24  perimeter_worst                          569 non-null    float64
```

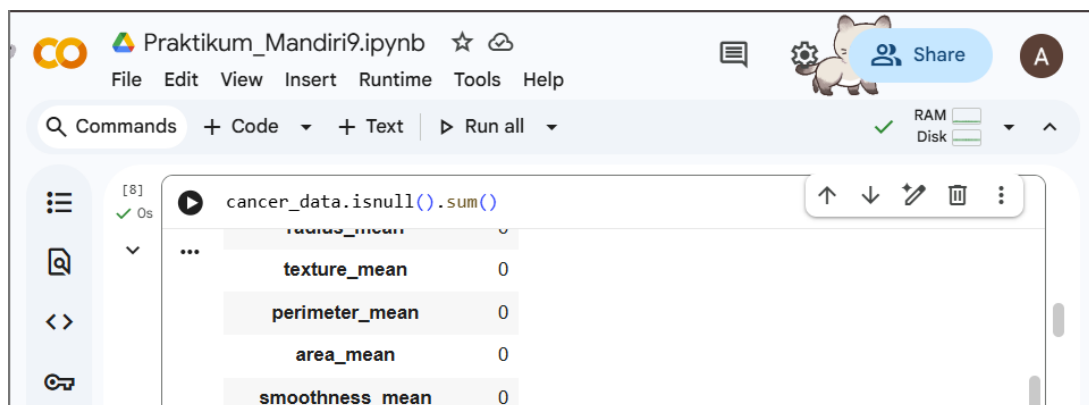
7. cancer_data.info()

Menampilkan informasi lengkap tentang dataset cancer_data,

seperti jumlah baris dan kolom, tipe data setiap kolom,

jumlah data non-null pada tiap kolom, serta penggunaan memori.

Fungsi ini membantu memahami struktur dataset sebelum dilakukan analisis lebih lanjut.



```
cancer_data.isnull().sum()

radius_mean      0
...
texture_mean      0
perimeter_mean    0
area_mean         0
smoothness_mean   0
```

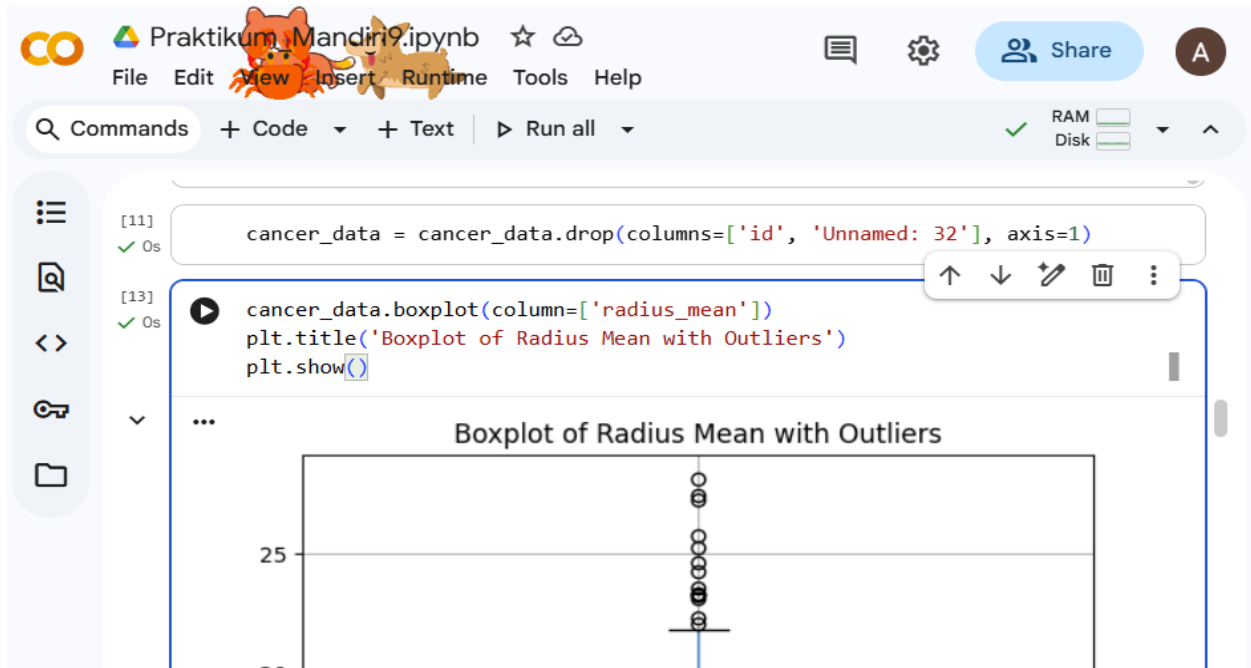
8. cancer_data.isnull().sum()

Mengecek jumlah nilai kosong (missing value) pada setiap kolom dalam dataset.

Output akan menunjukkan kolom mana saja yang memiliki data hilang dan berapa jumlahnya.

Langkah ini penting sebelum melakukan preprocessing, karena data kosong perlu ditangani

agar tidak menyebabkan error atau mengurangi akurasi model.



9. `cancer_data = cancer_data.drop(columns=['id', 'Unnamed: 32'], axis=1)`

Menghapus kolom 'id' dan 'Unnamed: 32' dari dataset karena kedua kolom ini

tidak memiliki peran penting dalam proses analisis maupun pemodelan.

'id' hanya berfungsi sebagai identitas unik setiap data,

sedangkan 'Unnamed: 32' biasanya merupakan kolom kosong hasil ekspor data.

Dengan menghapusnya, dataset menjadi lebih bersih dan siap untuk tahap preprocessing berikutnya.

10. `cancer_data.boxplot(column=['radius_mean'])`

`plt.title('Boxplot of Radius Mean with Outliers')`

`plt.show()`

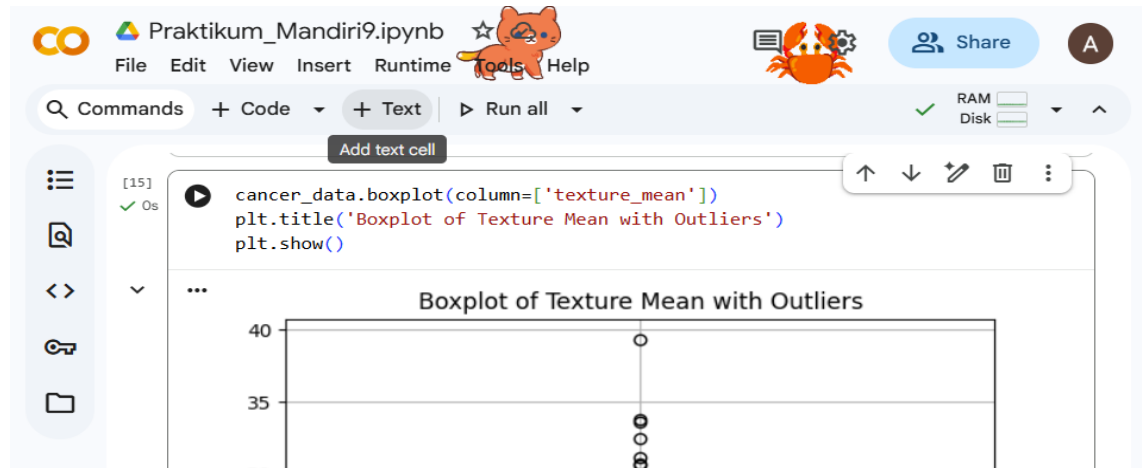
Membuat boxplot untuk kolom 'radius_mean' pada dataset.

Boxplot ini digunakan untuk menampilkan distribusi data sekaligus

mendeteksi apakah terdapat outlier (nilai pencilan) pada fitur tersebut.

Outlier biasanya terlihat sebagai titik di luar garis whisker,

dan informasi ini penting untuk keputusan preprocessing seperti normalisasi atau penghapusan outlier.



```
11. cancer_data.boxplot(column=['texture_mean'])
```

```
plt.title('Boxplot of Texture Mean with Outliers')
```

```
plt.show()
```

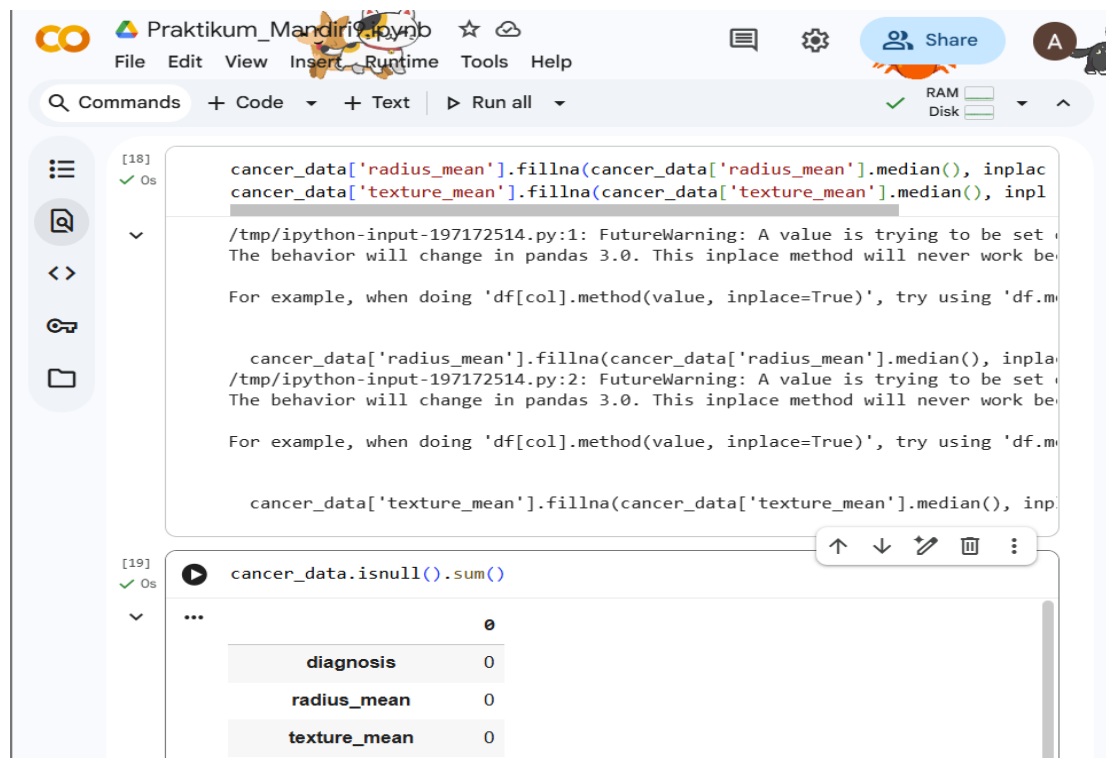
Membuat boxplot untuk kolom 'texture_mean' pada dataset.

Visualisasi ini digunakan untuk melihat persebaran data serta

mengetahui apakah ada nilai outlier (pencilan) pada fitur tersebut.

Jika terdapat titik yang berada di luar garis whisker, maka itu menandakan outlier

yang mungkin perlu dianalisis atau ditangani pada tahap preprocessing.



```
[18] ✓ Os
cancer_data['radius_mean'].fillna(cancer_data['radius_mean'].median(), inplace=True)
cancer_data['texture_mean'].fillna(cancer_data['texture_mean'].median(), inplace=True)

/tmp/ipython-input-197172514.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series consisting of zero or more existing elements. This behavior will change in pandas 3.0. This inplace method will never work because it changes the original object.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df[col] = df[col].method(value)' instead.

cancer_data['radius_mean'].fillna(cancer_data['radius_mean'].median(), inplace=True)
/tmp/ipython-input-197172514.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series consisting of zero or more existing elements. This behavior will change in pandas 3.0. This inplace method will never work because it changes the original object.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df[col] = df[col].method(value)' instead.

cancer_data['texture_mean'].fillna(cancer_data['texture_mean'].median(), inplace=True)

[19] ✓ Os
cancer_data.isnull().sum()

...
```

	0
diagnosis	0
radius_mean	0
texture_mean	0

12. `cancer_data['radius_mean'].fillna(cancer_data['radius_mean'].median(), inplace=True)`

`cancer_data['texture_mean'].fillna(cancer_data['texture_mean'].median(), inplace=True)`

Mengisi nilai kosong (missing values) pada kolom 'radius_mean' dan 'texture_mean'

dengan nilai median masing-masing kolom.

Median dipilih karena lebih robust terhadap outlier dibandingkan mean,

sehingga nilai yang diisi tidak akan terpengaruh oleh pencilan.

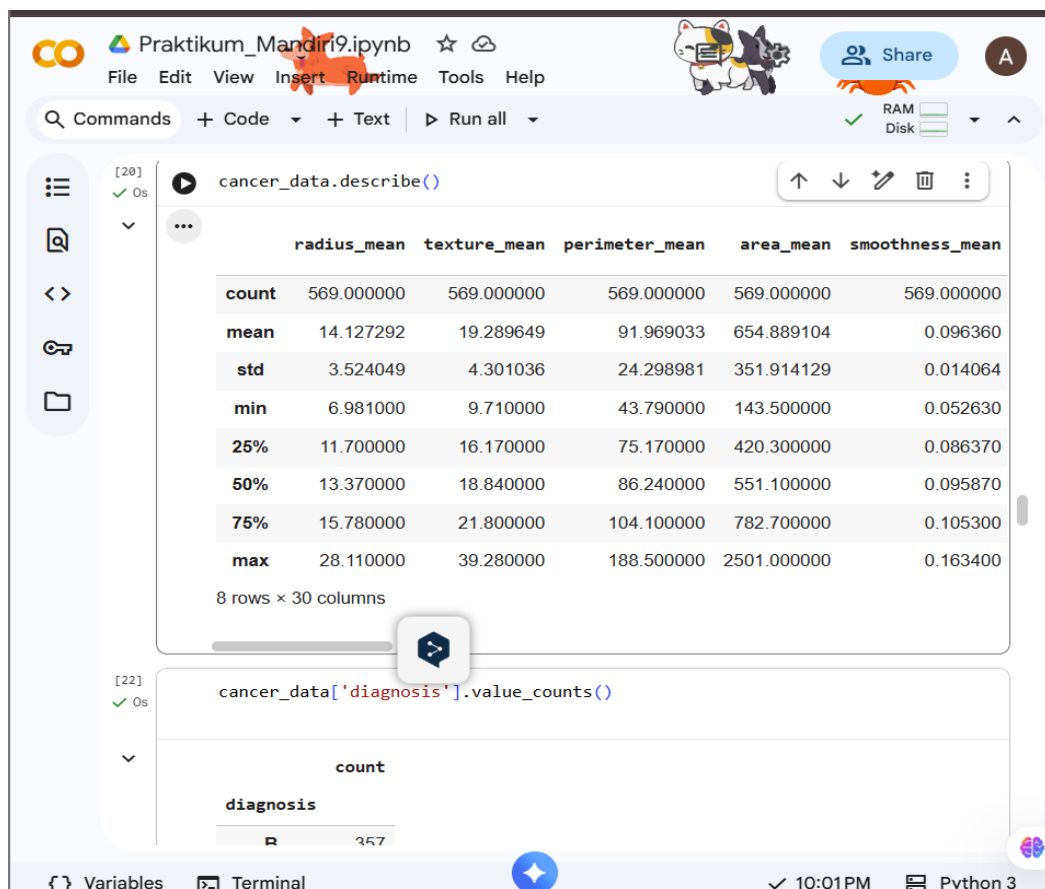
Parameter `inplace=True` membuat perubahan langsung diterapkan pada DataFrame tanpa membuat salinan baru.

13. `cancer_data.isnull().sum()`

Mengecek jumlah nilai kosong (missing values) pada setiap kolom dalam dataset.

Output akan menampilkan berapa banyak nilai kosong yang masih ada per kolom,

sehingga kita bisa mengetahui apakah perlu dilakukan proses pembersihan data (data cleaning) lanjutan.



14. `cancer_data.describe()`

- # Menampilkan statistik deskriptif dari semua kolom numerik dalam dataset,
- # seperti mean, standard deviation (std), nilai minimum, maksimum, serta nilai kuartil.
- # Perintah ini berguna untuk memahami karakteristik dasar data,
- # termasuk sebaran data, pusat data (mean/median), dan apakah ada nilai ekstrem (outlier).

15. `cancer_data['diagnosis'].value_counts()`

- # Menghitung jumlah kemunculan setiap kategori pada kolom 'diagnosis'.
- # Kolom 'diagnosis' biasanya berisi label klasifikasi seperti:
- # M = Malignant (ganas) dan B = Benign (jinak).
- # Perintah ini digunakan untuk melihat distribusi kelas dalam dataset,
- # apakah seimbang atau condong ke salah satu kelas,
- # yang penting untuk menentukan apakah dataset berpotensi mengalami imbalance.


```
[24] ✓ 0s
cancer_data.groupby('diagnosis').size()

diagnosis
B      357
M      212

dtype: int64
```

16. `cancer_data.groupby('diagnosis').size()`

- # Mengelompokkan data berdasarkan nilai pada kolom 'diagnosis'
- # lalu menghitung jumlah baris pada masing-masing kelompok.
- # Hasilnya sama seperti `value_counts()`, yaitu menunjukkan
- # berapa banyak data dengan diagnosis 'M' (ganas) dan 'B' (jinak).
- # Perintah ini sering digunakan untuk memastikan distribusi kelas,
- # apakah dataset seimbang atau memiliki ketimpangan (imbalance).

```
[25] ✓ 1s
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(20,15))
axes = axes.flatten()

sns.countplot(x='diagnosis', data=cancer_data, ax=axes[0])
axes[0].set_title('Diagnosis')

sns.histplot(cancer_data['radius_mean'], kde=True, ax=axes[1])
axes[1].set_title('Radius Mean')

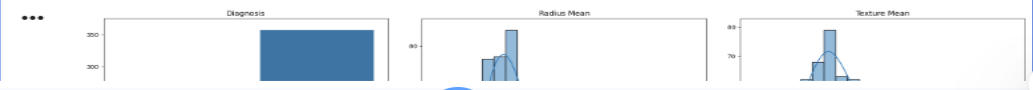
sns.histplot(cancer_data['texture_mean'], kde=True, ax=axes[2])
axes[2].set_title('Texture Mean')

sns.histplot(cancer_data['perimeter_mean'], kde=True, ax=axes[3])
axes[3].set_title('Perimeter Mean')

sns.histplot(cancer_data['area_mean'], kde=True, ax=axes[4])
axes[4].set_title('Area Mean')

sns.histplot(cancer_data['smoothness_mean'], kde=True, ax=axes[5])
axes[5].set_title('Smoothness Mean')

plt.tight_layout()
plt.show()
```



Variables Terminal 10:01 PM Python 3

17. # Membuat 6 visualisasi dalam satu figure untuk melihat distribusi data.

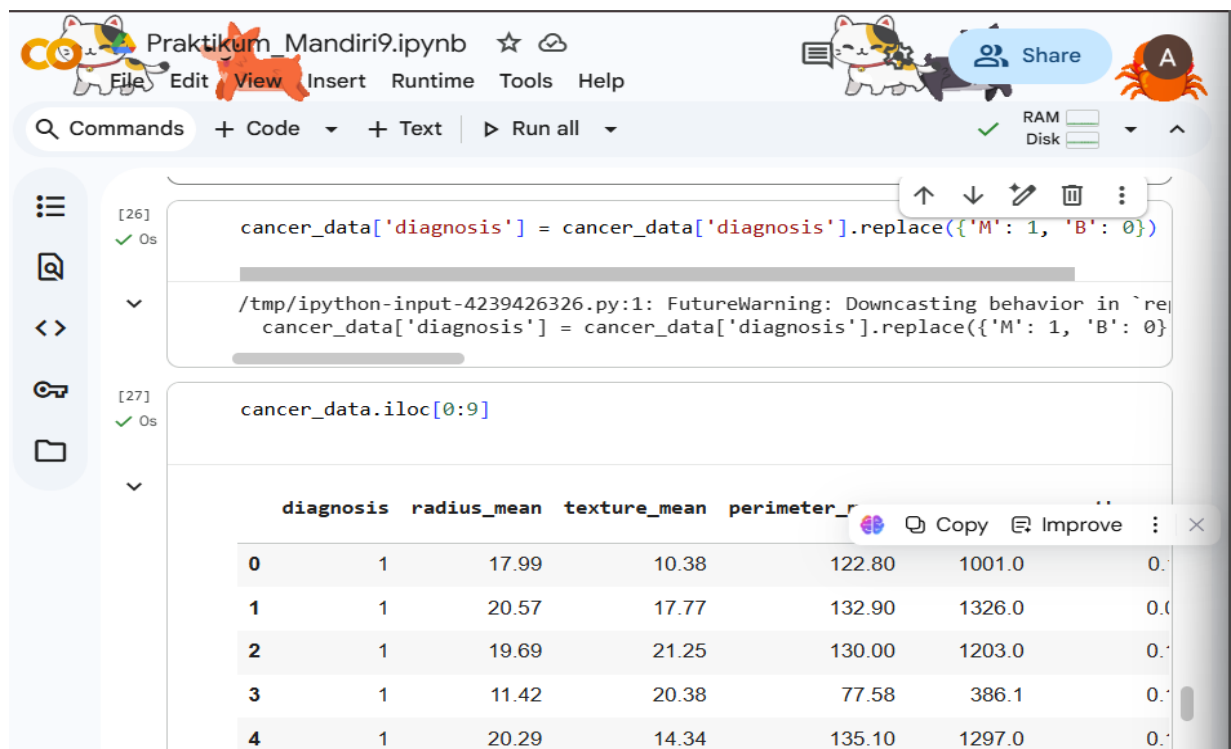
Grafik pertama (countplot) menampilkan jumlah kasus diagnosis (M dan B).

Grafik lainnya (histplot) menunjukkan distribusi masing-masing fitur numerik

seperti radius_mean, texture_mean, perimeter_mean, area_mean, dan smoothness_mean.

Visualisasi ini membantu memahami persebaran data dan karakteristik fitur

sebelum dilakukan proses modeling.



The screenshot shows a Jupyter Notebook titled 'Praktikum_Mandiri9.ipynb'. The code cell [26] contains the following Python code:

```
cancer_data['diagnosis'] = cancer_data['diagnosis'].replace({'M': 1, 'B': 0})
```

A warning message is displayed below the code: `/tmp/ipython-input-4239426326.py:1: FutureWarning: Downcasting behavior in `repl``. The code cell [27] contains the following Python code:

```
cancer_data.iloc[0:9]
```

The output of the code cell [27] is a table showing the first 9 rows of the cancer_data dataset. The table has 7 columns: diagnosis, radius_mean, texture_mean, perimeter_r, area_r, smoothness_r, and compactness_r. The first 5 rows are visible:

	diagnosis	radius_mean	texture_mean	perimeter_r	area_r	smoothness_r
0	1	17.99	10.38	122.80	1001.0	0.1096
1	1	20.57	17.77	132.90	1326.0	0.1613
2	1	19.69	21.25	130.00	1203.0	0.1853
3	1	11.42	20.38	77.58	386.1	0.2019
4	1	20.29	14.34	135.10	1297.0	0.1735

18. cancer_data['diagnosis'] = cancer_data['diagnosis'].replace({'M': 1, 'B': 0})

Mengubah nilai pada kolom 'diagnosis'

Nilai 'M' (ganas) diganti menjadi 1, dan 'B' (jinak) diganti menjadi 0.

Tujuannya supaya data bisa diproses oleh algoritma machine learning

yang lebih mudah bekerja dengan angka daripada huruf.

19. cancer_data.iloc[0:9]

Mengambil baris pertama hingga baris ke-9 dari dataset 'cancer_data'.

'iloc' digunakan untuk mengakses data berdasarkan posisi baris dan kolom (index).

Hasilnya menampilkan 9 baris pertama lengkap dengan semua kolomnya.

```
[30] X = cancer_data.drop(columns=['diagnosis'])
      Y = cancer_data['diagnosis']

[31] X.head()

...
      radius_mean  texture_mean  perimeter_mean  area_mean  smoothness_mean  comp:
0      17.99      17.77      132.90      1326.0      0.08474
1      20.57      21.25      130.00      1203.0      0.10960
2      19.69      20.38      77.58      386.1      0.14250
3      11.42      14.34      135.10      1297.0      0.10030
4      20.29      14.34      135.10      1297.0      0.10030
5 rows x 30 columns

[32] Y.head()

...
      diagnosis
0      1
1      1
```

20. X = cancer_data.drop(columns=['diagnosis'])

Y = cancer_data['diagnosis']

Memisahkan fitur dan target dari dataset.

'X' berisi semua kolom kecuali 'diagnosis' (fitur/input untuk model).

'Y' berisi kolom 'diagnosis' saja (target/output yang ingin diprediksi).

Pemisahan ini penting sebelum melatih model machine learning.

21. X.head()

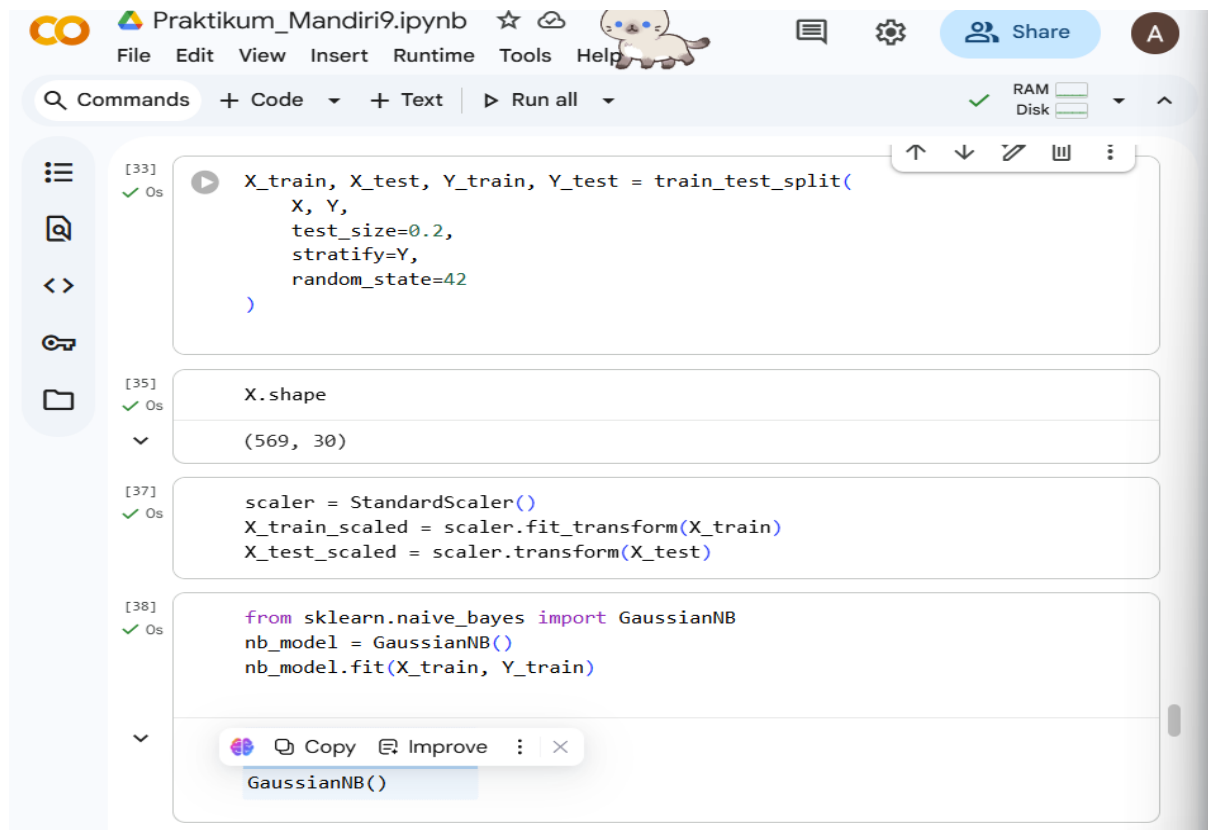
Menampilkan 5 baris pertama dari dataset 'X' (fitur).

Berguna untuk melihat sekilas isi data dan memastikan kolom-kolomnya benar.

22. Y.head()

Menampilkan 5 baris pertama dari dataset 'Y' (target/label).

Berguna untuk memastikan nilai target (diagnosis) sudah benar setelah dipisahkan.



The screenshot shows a Jupyter Notebook titled 'Praktikum_Mandiri9.ipynb'. The interface includes a top menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. On the left is a sidebar with icons for file operations. The main area displays four code cells, each with a status indicator (green checkmark) and execution time (0s). The first cell contains a `train_test_split` function call. The second cell shows the output of `X.shape` as `(569, 30)`. The third cell contains code for scaling the data using `StandardScaler`. The fourth cell contains code for importing `GaussianNB` and fitting the model. A tooltip for `GaussianNB()` is visible at the bottom of the fourth cell.

```
[33] ✓ 0s X_train, X_test, Y_train, Y_test = train_test_split(  
    X, Y,  
    test_size=0.2,  
    stratify=Y,  
    random_state=42  
)
```

```
[35] ✓ 0s X.shape  
      (569, 30)
```

```
[37] ✓ 0s scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

```
[38] ✓ 0s from sklearn.naive_bayes import GaussianNB  
nb_model = GaussianNB()  
nb_model.fit(X_train, Y_train)
```

GaussianNB()

23. X_train, X_test, Y_train, Y_test = train_test_split(

X, Y,

test_size=0.2,

stratify=Y,

random_state=42

)

```
# Membagi dataset menjadi data latih (train) dan data uji (test).  
  
# 'test_size=0.2' artinya 20% data dijadikan data uji, 80% untuk data latih.  
  
# 'stratify=Y' memastikan proporsi kelas pada Y (diagnosis) tetap sama di train dan test.  
  
# 'random_state=42' digunakan agar pembagian data bisa direproduksi.
```

24. X.shape

```
# Menampilkan ukuran dataset 'X' dalam bentuk (baris, kolom).  
  
# Baris menunjukkan jumlah sampel, kolom menunjukkan jumlah fitur.  
  
# Berguna untuk mengetahui seberapa besar dataset sebelum pemrosesan lebih lanjut.
```

25. scaler = StandardScaler()

```
X_train_scaled = scaler.fit_transform(X_train)  
  
X_test_scaled = scaler.transform(X_test)  
  
# Melakukan standarisasi fitur agar memiliki skala yang sama.  
  
# 'StandardScaler()' mengubah data sehingga rata-rata = 0 dan standar deviasi = 1.  
  
# 'fit_transform' diterapkan pada data latih untuk menghitung parameter skala dan menerapkannya.  
  
# 'transform' diterapkan pada data uji menggunakan parameter yang sama dari data latih.  
  
# Tujuannya agar algoritma machine learning lebih stabil dan akurat.
```

26. from sklearn.naive_bayes import GaussianNB

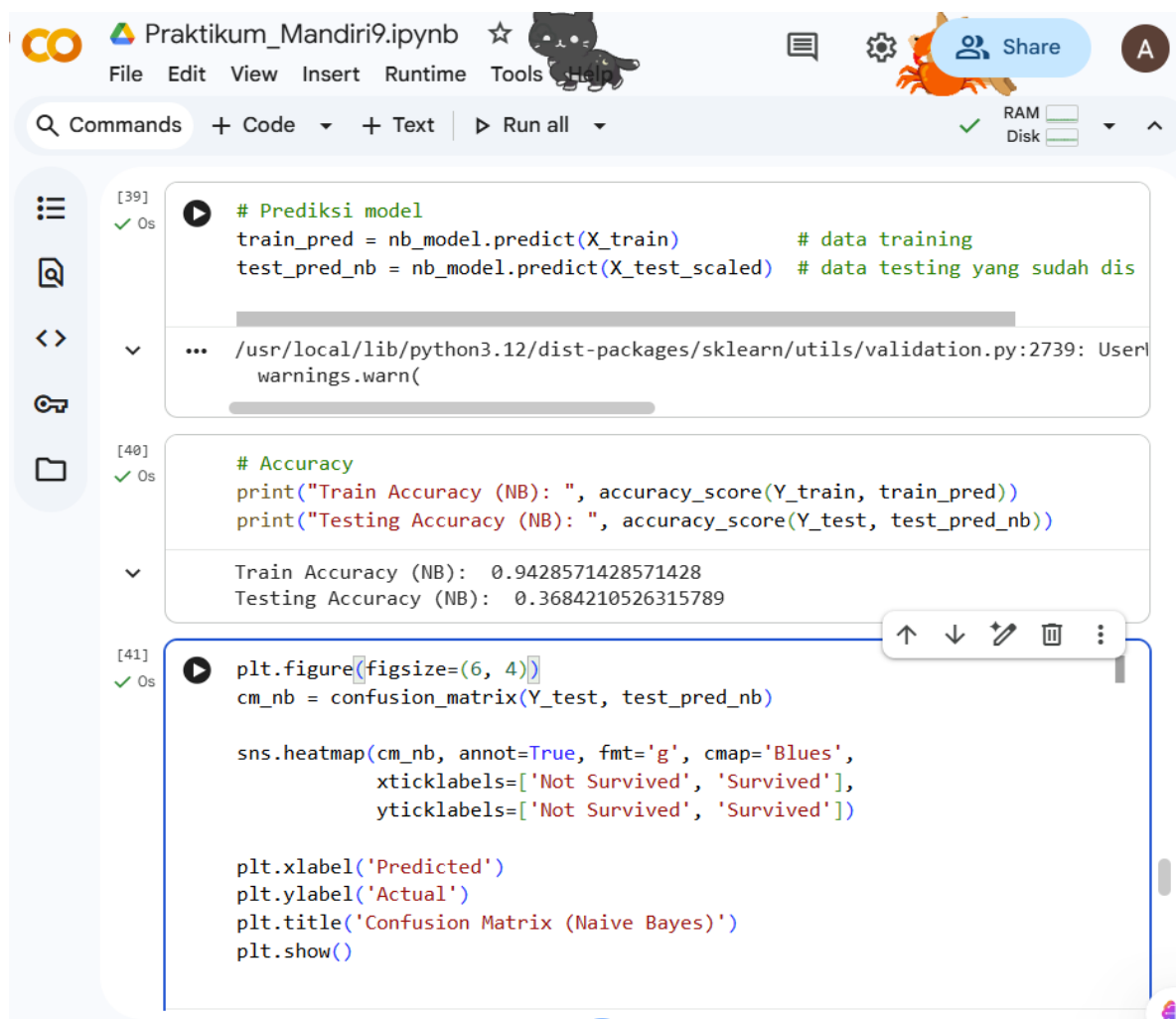
```
nb_model = GaussianNB()  
  
nb_model.fit(X_train, Y_train)
```

```
# Mengimpor dan membuat model Naive Bayes Gaussian.
```

'GaussianNB()' cocok untuk data numerik yang mengikuti distribusi normal.

'fit' digunakan untuk melatih model menggunakan data latih (X_train, Y_train).

Setelah ini, model siap digunakan untuk memprediksi data baru.



```
[39] # Prediksi model
train_pred = nb_model.predict(X_train) # data training
test_pred_nb = nb_model.predict(X_test_scaled) # data testing yang sudah dis

... /usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: UserWarning:

[40] # Accuracy
print("Train Accuracy (NB): ", accuracy_score(Y_train, train_pred))
print("Testing Accuracy (NB): ", accuracy_score(Y_test, test_pred_nb))

Train Accuracy (NB): 0.9428571428571428
Testing Accuracy (NB): 0.3684210526315789

[41] plt.figure(figsize=(6, 4))
cm_nb = confusion_matrix(Y_test, test_pred_nb)

sns.heatmap(cm_nb, annot=True, fmt='g', cmap='Blues',
            xticklabels=['Not Survived', 'Survived'],
            yticklabels=['Not Survived', 'Survived'])

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix (Naive Bayes)')
plt.show()
```

27. # Prediksi model

```
train_pred = nb_model.predict(X_train) # data training
```

```
test_pred_nb = nb_model.predict(X_test_scaled) # data testing yang sudah discale
```

Menggunakan model Naive Bayes yang sudah dilatih untuk membuat prediksi.

'train_pred' berisi prediksi untuk data latih, berguna untuk evaluasi performa awal.

'test_pred_nb' berisi prediksi untuk data uji yang sudah distandarisasi.

Prediksi ini nantinya bisa dibandingkan dengan nilai asli untuk mengukur akurasi model.

28. # Accuracy

```
print("Train Accuracy (NB): ", accuracy_score(Y_train, train_pred))
```

```
print("Testing Accuracy (NB): ", accuracy_score(Y_test, test_pred_nb))
```

Menghitung akurasi model Naive Bayes.

'accuracy_score' membandingkan prediksi dengan nilai asli.

'Train Accuracy' menunjukkan seberapa baik model memprediksi data latih.

'Testing Accuracy' menunjukkan seberapa baik model memprediksi data uji.

Akurasi adalah rasio prediksi yang benar terhadap total data.

29. # Membuat visualisasi confusion matrix untuk model Naive Bayes.

'confusion_matrix' membandingkan prediksi dengan nilai asli pada data uji.

'sns.heatmap' menampilkan matriks sebagai diagram warna dengan angka di tiap kotak.

Sumbu X = prediksi model, Sumbu Y = nilai asli.

Berguna untuk melihat jumlah prediksi benar dan salah per kelas.

The screenshot shows a Jupyter Notebook with two cells. The first cell, labeled [42], contains code to print a classification report for a Naive Bayes model. The output is a table with columns for precision, recall, f1-score, and support for each class, as well as overall accuracy, macro average, and weighted average. The second cell, labeled [43], contains code to perform 5-fold cross-validation using the accuracy scoring metric and print the mean and standard deviation of the scores.

```
[42] ✓ 0s
print("\nClassification Report (NB):")
print(classification_report(Y_test, test_pred_nb))

...

Classification Report (NB):
              precision    recall  f1-score   support

         0           0.00        0.00        0.00         72
         1           0.37        1.00        0.54         42

 accuracy          0.18        0.50        0.27        114
 macro avg          0.18        0.50        0.27        114
weighted avg          0.14        0.37        0.20        114

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

[43] ✓ 0s
from sklearn.model_selection import cross_val_score
cv_nb = cross_val_score(nb_model, X, Y, cv=5, scoring='accuracy')
print("\nNaive Bayes Cross Validation Accuracy (5-Fold):")
print("Scores:", cv_nb)
print("Mean Accuracy:", cv_nb.mean())
print("Std Deviation:", cv_nb.std())
```

```
30. print("\nClassification Report (NB):")
```

```
print(classification_report(Y_test, test_pred_nb))
```

```
# Menampilkan laporan klasifikasi untuk model Naive Bayes.
```

```
# 'classification_report' memberikan metrik: precision, recall, f1-score, dan support untuk tiap kelas.
```

```
# Berguna untuk mengevaluasi performa model lebih detail daripada sekadar akurasi.
```

```
# Dapat menunjukkan apakah model lebih baik pada salah satu kelas dibanding kelas lain.
```

```
31. from sklearn.model_selection import cross_val_score
```

```
cv_nb = cross_val_score(nb_model, X, Y, cv=5, scoring='accuracy')
```

```
print("\nNaive Bayes Cross Validation Accuracy (5-Fold):")
```

```
print("Scores:", cv_nb)
```



```
print("Mean Accuracy:", cv_nb.mean())

print("Std Deviation:", cv_nb.std())

# Melakukan cross-validation 5-fold untuk model Naive Bayes.

# 'cross_val_score' membagi data menjadi 5 bagian (fold), melatih model di 4 fold dan menguji di 1 fold secara bergantian.

# 'scoring="accuracy"' berarti evaluasi menggunakan akurasi.

# 'Scores' menampilkan akurasi tiap fold.

# 'Mean Accuracy' menunjukkan rata-rata akurasi dari semua fold.

# 'Std Deviation' menunjukkan seberapa bervariasi akurasi tiap fold.

# Tujuannya untuk mengevaluasi performa model secara lebih stabil dan mengurangi bias dari satu pembagian data.
```

LINK GITHUB UPLOAD TUGAS :

https://github.com/AliaMaisyarah14/PRAKTIKUM_ML/tree/7679164e2dbda8ffd64dd84737c92355498ff1dd/Praktikum9-AliaMaisyarah/Praktikum9

